

IMPLEMENTATION OF THE I²C BUS **PROTOCOL USING VERILOG HDL**

PROJECT REPORT

Submitted by
Tejasswat Rajindu

Internship Organization
VVDN Technologies Pvt. Ltd.

Industry Mentor
Sarath M

Project Duration
May 2026 – June 2026

ABSTRACT

The Inter-Integrated Circuit (I²C) protocol is one of the most widely used serial communication protocols for interfacing microcontrollers, sensors, memory devices, and peripheral integrated circuits due to its simplicity and support for multiple devices on a shared bus. This project presents the design and implementation of a fully functional I²C communication system using Verilog HDL for FPGA-based applications.

The implemented design supports communication between two masters and two slaves over a shared open-drain SDA and SCL bus. Key features include multi-master arbitration, multi-slave addressing, start and stop condition detection, ACK/NACK generation and detection, bidirectional read and write transactions, and address matching. Separate finite state machines were developed for both the master and slave controllers to ensure protocol-compliant communication. Arbitration logic was implemented to detect bus contention and allow only one master to continue transmission while the other releases the bus.

The complete design was functionally verified using Xilinx Vivado through simulation of various communication scenarios, including single-master transactions, multi-master arbitration, write operations, and bidirectional read operations. The simulation results confirmed correct protocol behaviour and reliable data transfer between masters and slaves.

This project provided practical experience in digital design, finite state machine development, shared-bus communication, and hardware protocol implementation using Verilog HDL.

TABLE OF CONTENTS

Chapter 1: Introduction.....	4
1.1 Introduction	
1.2 Background	
1.3 Problem Statement	
1.4 Objectives	
1.5 Scope of the Project	
Chapter 2: Literature Review and Background.....	8
2.1 I ² C Bus Architecture	
2.2 Communication Process	
2.3 START and STOP Conditions	
2.4 Address Fram and Read/Write Bit	
2.5 ACK and NACK Mechanism	
2.6 Write Operation	
2.7 Read Operation	
2.8 Multi-Master Arbitration	
2.9 Open-Drain Communication	
2.10 Advantages and limitations of the I ² C Protocol	
Chapter 3: System Design and Implementation.....	17
3.1 Overall System Design	
3.2 Top Module	
3.3 Master Controller	
3.4 Slave Controller	
3.5 Master Finite State Machine	
3.6 Slave Finite State Machine	
3.7 Arbitration Logic	
3.8 Read and Write Operations	
3.9 Simulation Environment	
Chapter 4: Simulation Results and Verification.....	32
4.1 Introduction	
4.2 Write Transaction Verification	
4.3 Read Transaction Verification	
4.4 Multi-Master Arbitration Verification	
4.5 Multi-Slave Communication Verification	
4.6 START/STOP Detection and ACK/NACK Verification	
4.7 Functional Summary	
Chapter 5: Conclusion and Future Work.....	39
5.1 Conclusion	
5.2 Future Work	

CHAPTER 1: INTRODUCTION

1.1 Introduction

The Inter-Integrated Circuit (I²C) protocol is a synchronous serial communication protocol developed by Philips Semiconductors (now NXP Semiconductors) for communication between integrated circuits over short distances. It has become one of the most widely adopted communication standards in embedded systems due to its simplicity, low pin count, and ability to connect multiple devices using only two communication lines: the Serial Data (SDA) line and the Serial Clock (SCL) line.

I²C is commonly used to interface microcontrollers with peripheral devices such as sensors, EEPROMs, real-time clocks, display controllers, analog-to-digital converters, and other integrated circuits. The protocol supports multiple master and multiple slave devices connected to the same bus while providing mechanisms for device addressing, synchronization, acknowledgment, and arbitration.

With the increasing use of Field Programmable Gate Arrays (FPGAs) in digital system design, implementing communication protocols such as I²C in hardware provides valuable insight into finite state machine design, timing control, bus arbitration, and digital communication techniques. Hardware implementation also enables designers to customize the protocol according to application-specific requirements while achieving high performance and deterministic operation.

This project presents the design and implementation of a fully functional I²C controller using Verilog HDL. The implemented design supports multiple masters and multiple slaves communicating over a shared open-drain bus. Features such as arbitration handling, start and stop condition detection, ACK/NACK generation, bidirectional read and write transactions, and address matching have been incorporated to closely follow the I²C protocol specification. The complete design has been functionally verified using simulation in Xilinx Vivado.

1.2 Background

The Inter-Integrated Circuit (I²C) protocol is a synchronous, serial communication protocol introduced by Philips Semiconductors in 1982 to facilitate communication between integrated circuits on the same printed circuit board (PCB). It is widely used in embedded systems because it requires only two communication lines, making it a simple and cost-effective solution for connecting multiple devices.

An I²C bus consists of two bidirectional lines: the Serial Data (SDA) line and the Serial Clock (SCL) line. The SDA line is responsible for transferring data between devices, while the SCL line provides the clock signal that synchronizes data transmission. Both lines operate using an open-drain configuration, requiring external pull-up resistors to maintain a logic high state when no device is actively driving the bus.

The protocol supports communication between one or more master devices and one or more slave devices. The master initiates communication by generating a START condition, transmitting the address of the intended slave along with a Read/Write (R/W) bit, and controlling the clock signal throughout the transaction. The addressed slave responds with an acknowledgment (ACK) and either receives or transmits data depending on the value of the R/W bit. Communication is terminated by the master using a STOP condition.

One of the key advantages of the I²C protocol is its support for multi-master communication. When multiple masters attempt to access the bus simultaneously, an arbitration mechanism ensures that only one master continues transmission while the others release the bus without corrupting the data. This arbitration is achieved by continuously monitoring the SDA line while transmitting data, allowing the protocol to resolve bus contention automatically.

Due to its simplicity, reliability, and widespread industry adoption, the I²C protocol is commonly used for interfacing microcontrollers, sensors, EEPROMs, real-time clocks, display controllers, analog-to-digital converters, and numerous other peripheral devices. Understanding and implementing the protocol in hardware provides valuable experience in digital design, finite state machine implementation, timing analysis, and communication protocol development.

1.3 Problem Statement

Serial communication protocols are fundamental to modern embedded systems, enabling reliable communication between processors and peripheral devices. While many commercial microcontrollers provide built-in I²C peripherals, implementing the protocol from scratch in hardware offers a deeper understanding of digital communication, timing synchronization, and finite state machine design.

Designing an I²C controller is challenging because it requires strict adherence to protocol timing while supporting features such as address recognition, acknowledgment generation, start and stop condition detection, bidirectional data transfer, and shared-bus communication. The complexity further increases in a multi-master environment, where multiple masters may attempt to access the bus simultaneously. In such cases, an

arbitration mechanism is required to ensure that only one master continues transmission while the others release the bus without causing data corruption.

The objective of this project is to design and implement a fully functional I²C controller using Verilog HDL on an FPGA. The implementation supports multiple masters and multiple slaves communicating over a shared open-drain bus. The controller incorporates arbitration logic, address matching, ACK/NACK handling, start and stop condition detection, and bidirectional read and write operations. The complete design is verified through simulation to ensure reliable and protocol-compliant communication under various operating conditions.

1.4 **Objectives**

The primary objective of this project is to design and implement a fully functional I²C communication system using Verilog HDL for FPGA-based applications. The project aims to develop a protocol-compliant controller capable of supporting reliable communication between multiple master and slave devices over a shared bus.

The specific objectives of this project are:

- To study the I²C communication protocol and understand its operating principles.
- To design and implement an I²C Master controller using Verilog HDL.
- To design and implement an I²C Slave controller using Verilog HDL.
- To support communication between two master devices and two slave devices.
- To implement start and stop condition detection.
- To implement 7-bit addressing with Read/Write (R/W) bit support.
- To implement ACK/NACK generation and detection.
- To support bidirectional data transfer through both read and write transactions.
- To implement multi-master arbitration for resolving simultaneous bus access.
- To model the open-drain behavior of the SDA and SCL lines.
- To verify the functionality of the complete design using Xilinx Vivado simulation.
- To validate the implementation through various test cases, including write operations, read operations, multi-master arbitration, and multi-slave communication.

1.5 Scope of the Project

The scope of this project is limited to the design, implementation, and functional verification of the I²C protocol using Verilog HDL for FPGA-based applications. The implementation focuses on developing a protocol-compliant communication system capable of supporting multiple masters and multiple slaves connected through a shared open-drain bus.

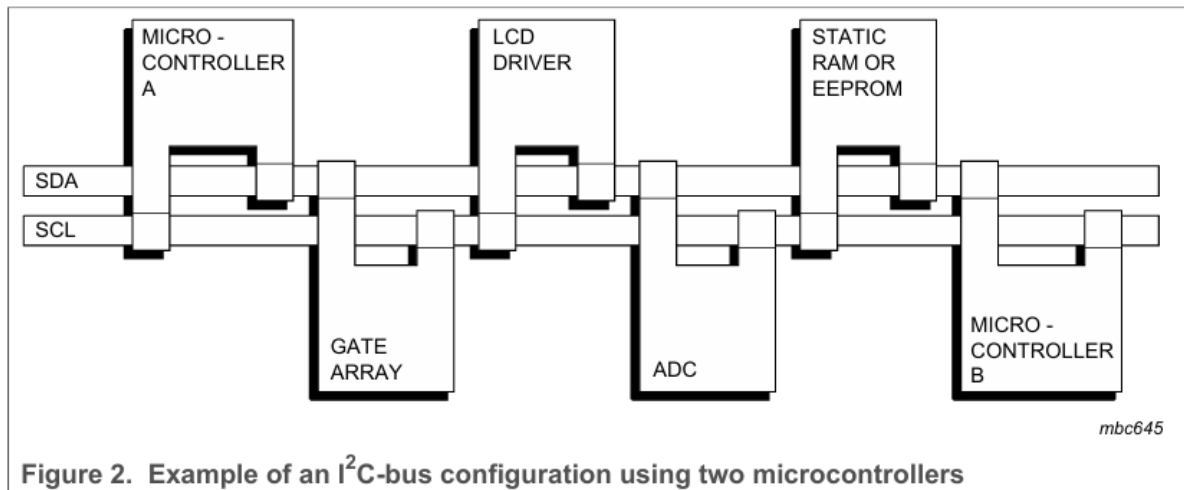
The implemented design includes multi-master arbitration, multi-slave communication, 7-bit addressing with Read/Write (R/W) bit support, start and stop condition detection, ACK/NACK generation and detection, bidirectional read and write transactions, and address matching. Separate finite state machines have been developed for both the master and slave controllers to ensure reliable communication in accordance with the I²C protocol. The functionality of the complete system has been verified through simulation using Xilinx Vivado under various operating scenarios, including single-master communication, multi-master arbitration, and bidirectional data transfer.

The current implementation is intended for functional verification and educational purposes. Features such as repeated START conditions, clock stretching, 10-bit addressing, multi-byte burst transfers, and hardware validation on a physical FPGA development board are beyond the scope of this project and may be considered as future enhancements.

Chapter 2: I²C Protocol

2.1 I²C Bus Architecture

The I²C (Inter-Integrated Circuit) bus is a synchronous, two-wire serial communication interface designed to enable communication between multiple integrated circuits using a minimal number of signal lines. Unlike many other communication protocols, I²C allows several master and slave devices to share the same communication bus, making it a compact and efficient solution for embedded systems.



The bus consists of two bidirectional communication lines:

Serial Data (SDA): Used for transmitting and receiving data between the master and slave devices.

Serial Clock (SCL): Used to synchronize data transmission. The clock signal is generated by the master device, while all connected slave devices synchronize their data transfers using this clock.

Both SDA and SCL operate using an open-drain (or open-collector) configuration. In this arrangement, devices can actively pull the communication lines to a logic low state but cannot drive them high. Instead, external pull-up resistors connected to the supply voltage maintain both lines at a logic high level whenever no device is pulling them low. This wired-AND configuration enables multiple devices to safely share the same communication bus without causing electrical conflicts.

An I²C system may contain one or more master devices and one or more slave devices connected to the same pair of communication lines. The master device initiates communication, generates the clock signal, selects the desired slave using its unique address, and controls the overall data transfer. Slave devices respond only when their assigned address is received and either transmit or receive data depending on the requested operation.

2.2 Communication Process

Communication in the I²C protocol is always initiated by a master device. Before transmitting data, the master checks whether the communication bus is idle by ensuring that both the SDA and SCL lines are in the logic high state. Once the bus becomes available, the master generates a START condition to indicate the beginning of a new communication session.

After generating the START condition, the master transmits the 7-bit address of the target slave followed by a single Read/Write (R/W) bit. The R/W bit determines the direction of data transfer. A logic '0' indicates a write operation, where the master sends data to the slave, while a logic '1' indicates a read operation, where the slave transmits data back to the master.

Upon receiving the address and R/W bit, all slave devices compare the received address with their own configured address. Only the slave with the matching address responds by pulling the SDA line low during the acknowledgment clock pulse, generating an ACK signal. All other slave devices ignore the remainder of the transaction.

Once the address has been acknowledged, data transfer begins. During a write operation, the master transmits one byte of data at a time while the addressed slave acknowledges each received byte. During a read operation, the addressed slave places data onto the SDA line while the master generates the clock pulses and acknowledges the received data.

After the required data has been transferred, the master terminates the communication by generating a STOP condition, releasing both SDA and SCL to the logic high state. The bus then becomes available for another communication session initiated by the same or a different master device.

2.3 Start and Stop Conditions

Every I²C communication transaction begins with a START condition and ends with a STOP condition. These conditions allow all devices connected to the bus to identify the beginning and end of a communication session. Since multiple masters and slaves share the same SDA and SCL lines, the START and STOP conditions play a critical role in coordinating bus access and maintaining reliable communication.

A START condition is generated by the master by pulling the SDA line from a logic high state to a logic low state while the SCL line remains high. This transition signals all connected slave devices that a new transmission is about to begin. Once the START condition has been generated, the master takes control of the bus and begins transmitting the slave address followed by the Read/Write (R/W) bit.

A STOP condition is generated by the master by allowing the SDA line to transition from a logic low state to a logic high state while the SCL line remains high. This indicates that the current communication has been completed and the bus is released for future transactions. After the STOP condition, both SDA and SCL return to their idle high state.

The I²C protocol uniquely identifies these conditions by monitoring transitions on the SDA line while SCL remains high. A falling edge on SDA during SCL high represents a START condition, whereas a rising edge on SDA during SCL high represents a STOP condition. Any transitions on SDA while SCL is low are interpreted as data bits rather than control signals.

In the proposed Verilog implementation, dedicated start and stop detection logic is incorporated within the slave controller to monitor the shared communication bus continuously. This enables the slave devices to recognize the beginning and end of each communication session and synchronize their state machines with the master's transmission.

2.4 Address Frame and Read/Write Bit

In the I²C protocol, every communication transaction begins with the transmission of an address frame immediately after the START condition. The address frame enables the master to select the intended slave device from all devices connected to the shared communication bus.

The standard I²C protocol uses a 7-bit slave address followed by a single Read/Write (R/W) bit, forming an 8-bit address frame. The first seven bits uniquely identify the target slave, while the least significant bit specifies the direction of data transfer.

The R/W bit has the following functions:

R/W = 0: Indicates a write operation, where the master transmits data to the addressed slave.

R/W = 1: Indicates a read operation, where the addressed slave transmits data to the master.

After the complete address frame has been transmitted, the addressed slave compares the received 7-bit address with its own configured address. If the addresses match, the slave responds by generating an Acknowledgment (ACK) during the ninth clock pulse. If no slave recognizes the transmitted address, no acknowledgment is generated, allowing the master to terminate the communication or attempt another transaction.

In the implemented design, the slave controller stores the received address frame and performs address matching by comparing only the upper seven bits with its predefined slave address. The least significant bit is interpreted as the Read/Write control bit, determining whether the slave enters receive mode or transmit mode.

2.5 Acknowledgment (ACK) and Not Acknowledgment (NACK)

The I²C protocol incorporates an acknowledgment mechanism to ensure reliable communication between the master and slave devices. After every 8-bit byte is transmitted, an additional clock pulse, known as the ninth clock pulse, is used for acknowledgment. This mechanism allows the receiving device to indicate whether the transmitted byte has been successfully received.

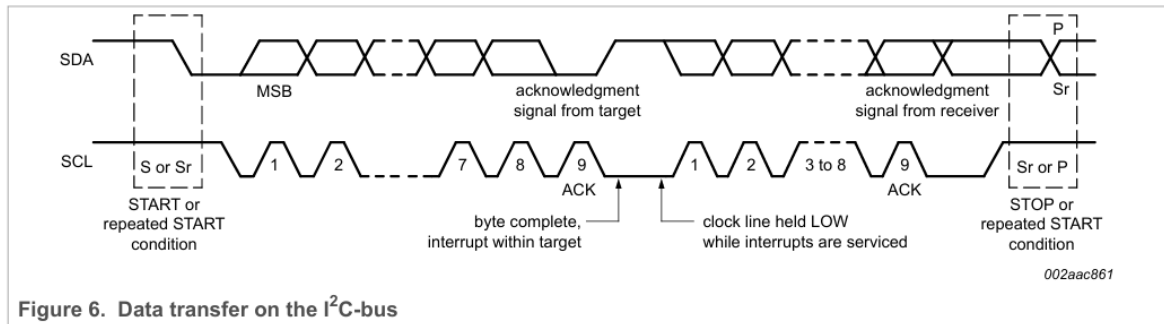
An Acknowledgment (ACK) is generated by the receiving device by pulling the SDA line low during the ninth clock pulse while the transmitter releases the SDA line. A low logic level on SDA during this clock pulse informs the transmitter that the data or address has been received correctly and that communication can continue.

A Not Acknowledgment (NACK) is generated by leaving the SDA line in the logic high state during the ninth clock pulse. A NACK indicates that the transmitted byte was not accepted or that the receiver is unable or unwilling to continue communication. Depending on the situation, the master may terminate the communication by issuing a STOP condition or initiate another transaction.

During a write operation, the slave generates the ACK signal after successfully receiving both the address frame and each data byte transmitted by the master. During a read operation, the master generates the ACK after receiving each byte from the slave if additional data is required. After the final byte has been received, the master transmits a NACK to indicate the end of the data transfer before generating the STOP condition.

In the proposed Verilog implementation, ACK generation and detection are incorporated into both the master and slave controllers. The slave generates an ACK after recognizing its address and after receiving valid data during write operations, while the master monitors the SDA line during the acknowledgment clock pulse to determine whether communication should continue.

Figure illustrates the ACK and NACK timing during the ninth clock pulse of an I²C transaction.



2.6 Write Operation

A write operation is performed when the master transmits data to a selected slave device. The transaction begins with the master generating a START condition, followed by the transmission of the 7-bit slave address and a Read/Write (R/W) bit set to logic '0', indicating a write request.

All slave devices connected to the bus receive the transmitted address and compare it with their configured address. The slave whose address matches the received address responds by generating an Acknowledgment (ACK) during the ninth clock pulse, while all other slave devices remain inactive.

After receiving the acknowledgment, the master transmits the data byte to the addressed slave. The slave samples each data bit on the rising edge of the clock signal and stores the received data internally. Upon successful reception of the complete data byte, the slave again generates an ACK signal to indicate successful data reception.

If additional data bytes are to be transmitted, the master continues sending data bytes, each followed by an acknowledgment from the slave. Once all required data has been transferred, the master generates a STOP condition, releasing the communication bus and completing the transaction.

The sequence of a standard I²C write operation is shown below:

START → Slave Address + Write (R/W = 0) → ACK → Data Byte → ACK → STOP

In the proposed implementation, the master controller is responsible for generating the START and STOP conditions, transmitting the address and data bytes, and monitoring the acknowledgment signals. The slave controller performs address matching, acknowledges the received address, stores the transmitted data, and generates the required ACK signals to complete the write transaction successfully.

2.7 Read Operation

A read operation is performed when the master requests data from a selected slave device. The communication begins with the master generating a START condition, followed by the transmission of the 7-bit slave address and a Read/Write (R/W) bit set to logic '1', indicating a read request.

After receiving the address frame, all slave devices compare the transmitted address with their configured addresses. The slave whose address matches responds by generating an Acknowledgment (ACK), indicating that it is ready to transmit data. Once the acknowledgment has been received, the addressed slave takes control of the SDA line and begins transmitting the requested data byte while the master continues generating the clock signal on the SCL line.

The master samples each transmitted bit on the rising edge of the clock signal and reconstructs the complete data byte. After receiving the data byte, the master generates an ACK if additional data bytes are required. If the received byte is the final byte of the transaction, the master generates a Not Acknowledgment (NACK) to indicate the end of the data transfer. The communication is then terminated by the master through the generation of a STOP condition.

The sequence of a standard I²C read operation is shown below:

START → Slave Address + Read (R/W = 1) → ACK → Data Byte → ACK/NACK → STOP

In the proposed implementation, the master controller enters a receive state after receiving the address acknowledgment for a read request. The addressed slave loads the requested data into its transmit register and drives the SDA line to transmit the data byte. The master samples the incoming data, stores it in its receive register, and generates the appropriate acknowledgment before completing the transaction with a STOP condition.

2.8 Multi-Master Arbitration

One of the key features of the I²C protocol is its support for multiple master devices sharing the same communication bus. Since more than one master can initiate communication, there is a possibility that two or more masters may attempt to access the bus simultaneously. To prevent data corruption, the I²C protocol employs a hardware-based arbitration mechanism that determines which master gains control of the bus.

Arbitration is performed during address and data transmission while the SCL line remains high. Each master continuously monitors the state of the SDA line while transmitting its own data. Since the SDA line operates in an open-drain configuration, a logic '0' is dominant over a logic '1'. Consequently, if a master attempts to transmit a logic '1' but

detects a logic '0' on the SDA line, it concludes that another master is transmitting a dominant bit and immediately loses arbitration.

Upon detecting arbitration loss, the losing master releases both the SDA and SCL lines and enters an idle or waiting state, allowing the winning master to complete the ongoing transaction without interruption. Once the communication has ended and the bus becomes free, the losing master may attempt to initiate a new transmission.

For example, consider two masters transmitting different addresses simultaneously. If Master 1 transmits a logic '0' while Master 2 transmits a logic '1' during the same clock cycle, the SDA line remains at logic '0' due to the open-drain bus configuration. Master 2 detects that the observed bus value differs from the value it transmitted and immediately withdraws from the communication, while Master 1 continues transmitting without any disruption.

In the proposed implementation, arbitration logic has been incorporated into the master controller by comparing the transmitted data bit with the actual state of the SDA line during every clock cycle. If the transmitted bit is logic '1' and the observed SDA line is logic '0', the controller asserts an arbitration loss signal, releases the communication bus, and waits until the bus becomes available before attempting another transaction.

2.9 Open-Drain Communication

The I²C protocol employs an open-drain (or open-collector) bus architecture for both the Serial Data (SDA) and Serial Clock (SCL) lines. Unlike conventional digital outputs that actively drive both logic high and logic low levels, an open-drain output can only drive the bus to a logic low state. A logic high is achieved by releasing the bus, allowing an external pull-up resistor to raise the line to the supply voltage.

This operating principle enables multiple devices to share the same communication bus without causing electrical conflicts. Since no device actively drives the bus to a logic high level, two or more devices can safely be connected to the same SDA and SCL lines. If any device pulls the line low, the bus assumes a logic low state regardless of the state of the other devices. Consequently, the bus behaves as a wired-AND connection, where a logic '0' is dominant over a logic '1'.

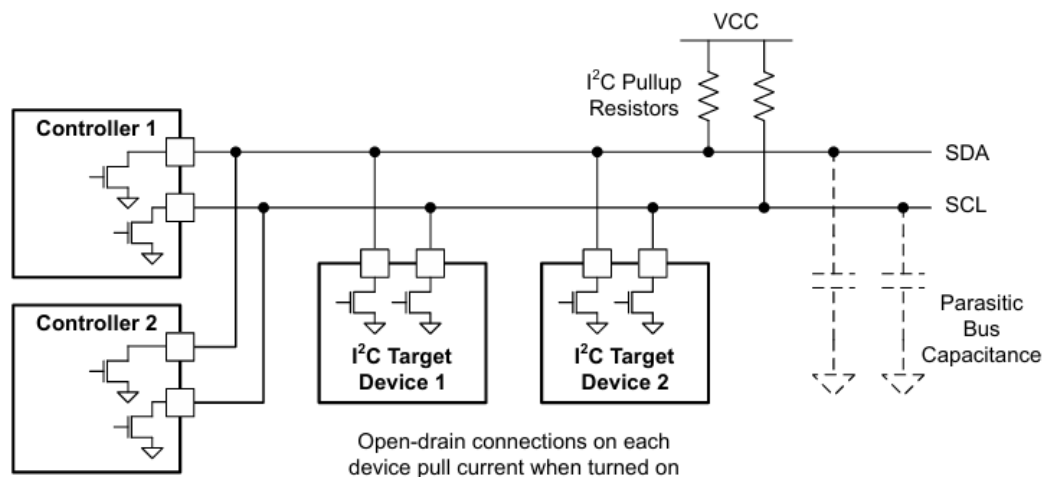
The open-drain configuration is fundamental to several important features of the I²C protocol. It enables multiple masters to communicate over the same bus, allows arbitration to occur without data corruption, and permits slave devices to acknowledge transmitted data by pulling the SDA line low during the acknowledgment clock pulse.

In the proposed Verilog implementation, the open-drain behavior of the SDA and SCL lines is modeled using tri-state outputs. A device actively drives the line only when transmitting a logic low. Otherwise, it releases the line by placing it in a high-impedance state, allowing the pull-up resistor to restore the logic high level. This behavior is implemented using the following Verilog statements:

```
assign sda = sda_drive ? 1'b0 : 1'bz;
assign scl = scl_drive ? 1'b0 : 1'bz;
```

This implementation accurately represents the electrical characteristics of the I²C bus and ensures reliable communication between multiple master and slave devices sharing the same communication lines.

Figure illustrates the open-drain configuration of the SDA and SCL lines with external pull-up resistors connected to the supply voltage.



2.10 Advantages and Limitations of the I²C Protocol

The I²C protocol has become one of the most widely adopted communication protocols in embedded systems due to its simplicity, flexibility, and efficient use of hardware resources. Its two-wire architecture significantly reduces the number of input/output pins required for communication, making it well suited for systems containing multiple peripheral devices.

One of the major advantages of the I²C protocol is its support for multiple master and multiple slave devices on a single communication bus. Each slave device is uniquely identified by its address, allowing several devices to communicate without requiring dedicated communication lines. The protocol also incorporates built-in acknowledgment and arbitration mechanisms, improving communication reliability and enabling safe bus sharing among multiple masters.

Another important advantage is the open-drain bus architecture, which allows multiple devices to safely share the SDA and SCL lines while preventing electrical contention. The protocol is widely supported by microcontrollers, sensors, memory devices, display modules, and numerous other integrated circuits, making it a standard communication interface in modern embedded systems.

Despite these advantages, the I²C protocol also has certain limitations. Compared to protocols such as SPI, I²C provides lower data transfer rates, making it less suitable for applications requiring high-speed communication. The use of pull-up resistors and the capacitive loading of the communication bus also limit the maximum bus length and operating speed. Furthermore, the protocol is half-duplex, meaning data transfer can occur in only one direction at a time.

Advantages	Limitations
Two-wire communication interface	Lower speed compared to SPI
Supports multiple masters and slaves	Limited bus length due to capacitance
Requires fewer I/O pins	Pull-up resistors are required
Built-in addressing and acknowledgment	Half-duplex communication
Supports arbitration for bus sharing	Performance decreases with increasing bus load
Simple and cost-effective implementation	Not suitable for very high-speed applications

Table 2.1 summarizes the major advantages and limitations of the I²C protocol.

The combination of simplicity, reliability, and widespread hardware support has made the I²C protocol one of the most popular serial communication standards for embedded and FPGA-based systems, particularly in applications where moderate data rates and efficient hardware utilization are more important than maximum communication speed.

Chapter 3: System Design and Implementation

3.1 Overall System Design

The proposed I²C communication system was designed and implemented using Verilog HDL with the objective of supporting communication between multiple master and multiple slave devices over a shared communication bus. The design follows the standard I²C protocol specification while incorporating additional functionality such as multi-master arbitration and bidirectional data transfer.

As illustrated in Figure 2.1, the system consists of two master controllers and two slave controllers connected through a common Serial Data (SDA) line and Serial Clock (SCL) line. The SDA and SCL lines are modeled as open-drain signals with pull-up resistors, allowing multiple devices to safely share the communication bus without electrical contention.

Each master controller operates independently and is capable of initiating communication by generating a START condition, transmitting the slave address along with the Read/Write (R/W) bit, performing data transfer, detecting acknowledgments, and generating a STOP condition. In situations where multiple masters attempt to access the bus simultaneously, arbitration logic ensures that only one master retains control while the others release the bus and wait until it becomes available again.

Each slave controller continuously monitors the communication bus for a START condition and receives the transmitted address. After comparing the received address with its configured slave address, the addressed slave either receives data from the master during write operations or transmits stored data back to the master during read operations. Slave devices also generate acknowledgment signals whenever valid addresses or data bytes are successfully received.

The overall design adopts a modular architecture in which each functional block is implemented as an independent Verilog module. This modular approach improves readability, simplifies debugging, and allows individual modules to be tested independently before integrating them into the complete system. The major modules developed for this project include the top module, master controller, slave controller, and testbench, each performing a specific function within the communication process.

The functionality of the complete system was verified through simulation using Xilinx Vivado. Multiple test scenarios, including write transactions, read transactions, multi-

slave communication, and simultaneous master access with arbitration, were performed to validate the correctness and reliability of the implemented design.

3.2 Top Module

The top module acts as the central integration unit of the proposed I²C communication system. It is responsible for interconnecting all master and slave controllers through a common communication bus and providing the infrastructure required for protocol-compliant data exchange. Rather than implementing communication logic directly, the top module coordinates the interaction between individual functional modules, enabling complete system-level operation.

The implemented system consists of two independent master controllers and two slave controllers connected to a shared Serial Data (SDA) line and Serial Clock (SCL) line. Each master receives its own control inputs, including a start signal, the target slave address, and the data to be transmitted. This arrangement allows either master to independently initiate a communication transaction while sharing the same physical bus. The shared-bus architecture also enables verification of simultaneous transmission attempts and arbitration between competing masters.

Each slave module is instantiated with a unique 7-bit slave address. During operation, both slaves continuously monitor the SDA and SCL lines for a valid START condition. Once communication begins, each slave captures the transmitted address frame and compares the received address with its internally configured address. Only the addressed slave participates in the transaction, while the remaining slave remains idle until a matching address is detected.

Communication between all devices takes place over shared SDA and SCL buses implemented as bidirectional (inout) signals. Since the I²C protocol uses an open-drain architecture, no device actively drives a logic high level onto the bus. Instead, devices either pull the communication line low or release it by entering a high-impedance state. External pull-up resistors connected to both SDA and SCL restore the bus to a logic high level whenever no device is driving it low. This implementation accurately models the electrical behavior of an actual I²C bus and allows multiple devices to safely share the communication lines.

The open-drain behavior of the communication bus is implemented using tri-state assignments within the master and slave modules. When a controller needs to transmit a logic low, it actively drives the corresponding line. Otherwise, it releases the line, allowing the pull-up resistor to maintain the idle high state. This approach ensures correct bus operation while preventing contention between multiple devices connected to the same communication lines.

The top module also provides the common platform required to evaluate protocol features such as address recognition, acknowledgment generation, arbitration, bidirectional read and write transactions, and simultaneous bus access by multiple masters. By integrating all functional blocks into a single design, it enables comprehensive functional verification of the complete communication system through simulation.

Figure 3.1 illustrates the RTL hierarchy of the implemented design, showing the interconnection between the top module, the two master controllers, the two slave controllers, and the shared SDA and SCL communication buses.

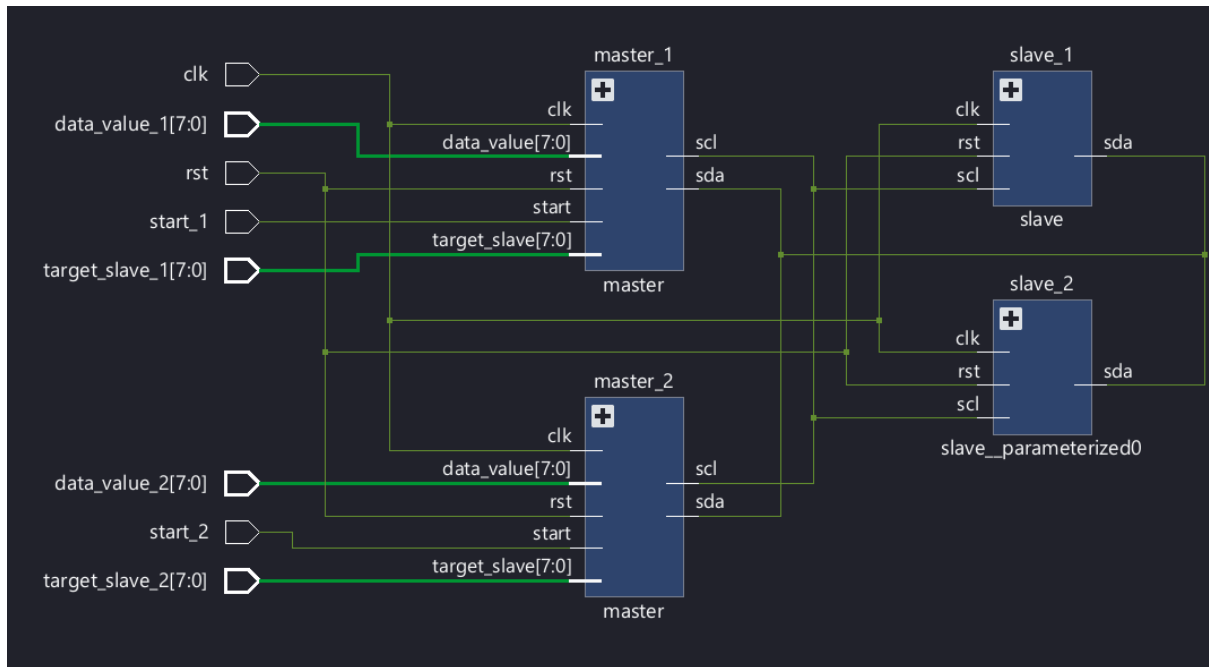


Figure 3.1

The corresponding RTL implementation instantiates the master and slave modules, connects them to the shared communication lines, and models the pull-up resistors required by the I²C protocol. This modular design approach improves readability, simplifies debugging, and allows each functional block to be developed and verified independently before system-level integration.

3.3 Master Controller

The master controller is the primary control unit responsible for initiating and managing all communication on the I²C bus. In accordance with the I²C protocol, only a master device is permitted to initiate communication, generate the clock signal, and terminate a transaction. The master controller developed in this project performs all protocol operations required for reliable communication with multiple slave devices while supporting multi-master arbitration and bidirectional data transfer.

The controller receives a system clock, reset signal, transaction start signal, target slave address, and the data byte to be transmitted. Based on these inputs, it generates the necessary control signals to establish communication over the shared SDA and SCL lines. Internally, the controller consists of several functional blocks, including the finite state machine (FSM), bit counter, address register, data register, acknowledgment detection logic, arbitration logic, and clock generation circuitry. Each block performs a specific task to ensure correct protocol operation.

When a transaction is initiated, the master first checks whether the communication bus is available before generating a START condition. It then transmits the slave address together with the Read/Write (R/W) bit, enabling the addressed slave to determine the direction of communication. During a write operation, the master serially transmits the data byte onto the SDA line while simultaneously generating the clock pulses on the SCL line. During a read operation, the master releases the SDA line after the address phase, allowing the addressed slave to transmit data while the master samples the incoming bits.

The master continuously monitors the acknowledgment bit transmitted by the slave after both the address frame and each data byte. Successful acknowledgments allow the communication to continue, whereas the absence of an acknowledgment results in termination of the transaction through the generation of a STOP condition. This mechanism ensures reliable communication and enables the controller to detect unavailable or non-responsive slave devices.

To support multiple masters on the same communication bus, arbitration logic is incorporated into the controller. During every transmitted bit, the master compares the value it intends to transmit with the actual state of the SDA line. If the master attempts to transmit a logic high while detecting a logic low on the bus, it recognizes that another master is transmitting a dominant bit and immediately declares arbitration loss. The controller then releases control of the communication bus and waits until the current transaction has completed before attempting another transmission.

The controller also generates the SCL timing required for communication. A clock divider derives the I²C clock from the system clock, ensuring that all data transmission, acknowledgment sampling, and arbitration decisions occur at the appropriate instants. Data shifting and sampling are synchronized with the generated clock edges to satisfy the timing requirements of the I²C protocol.

The master controller was designed using a modular approach, allowing the communication process to be divided into clearly defined functional blocks. This improves code readability, simplifies debugging, and enables future enhancements such as repeated

START conditions, clock stretching support, multi-byte transfers, and additional addressing modes without major modifications to the overall architecture.

The functional behavior of the master controller is governed by a finite state machine, which coordinates each stage of communication including idle operation, START generation, address transmission, acknowledgment handling, data transfer, arbitration monitoring, and STOP generation. The detailed operation of the finite state machine is discussed in Section 3.5.

3.4 Slave Controller

The slave controller is responsible for monitoring the shared I²C communication bus, recognizing transactions addressed to it, and participating in data transfer as required by the communication protocol. Unlike the master controller, the slave cannot initiate communication or generate the clock signal. Instead, it remains in an idle state while continuously observing the Serial Data (SDA) and Serial Clock (SCL) lines for a valid START condition generated by a master device.

Once a START condition is detected, the slave begins receiving the address frame transmitted by the master. The incoming serial bits are sampled using the clock generated by the master and stored in an internal address register. After the complete address frame has been received, the slave compares the received 7-bit address with its predefined address. If the addresses match, the slave acknowledges the transaction by pulling the SDA line low during the acknowledgment clock pulse. If the address does not match, the controller immediately returns to the idle state and ignores the remainder of the transaction.

The controller supports both write and read operations based on the value of the Read/Write (R/W) bit received as part of the address frame. During a write operation, the slave receives serial data from the master and stores it in an internal data register. During a read operation, the slave retrieves the stored data from its transmit register and serially transmits it back to the master while the master continues to generate the clock signal. This bidirectional communication enables the slave to function as both a data receiver and a data transmitter depending on the requested operation.

To ensure reliable synchronization with the communication bus, the slave continuously detects START and STOP conditions by monitoring transitions on the SDA line while the SCL line remains high. A falling edge on the SDA line during SCL high is interpreted as a START condition, whereas a rising edge indicates a STOP condition. These events control the operation of the slave finite state machine and ensure that communication begins and ends in accordance with the I²C specification.

The slave controller also incorporates acknowledgment generation logic. Whenever a valid address or data byte is successfully received, the controller temporarily drives the SDA line low during the ninth clock pulse to generate an ACK signal. At all other times, the SDA line remains in a high-impedance state, preserving the open-drain characteristics of the I²C bus and preventing contention with other devices.

Internally, the slave controller consists of several functional blocks, including the finite state machine, address register, data register, bit counter, address comparison logic, START/STOP detection circuitry, and SDA control logic. Together, these blocks enable the slave to correctly synchronize with the master's clock, receive or transmit data, generate acknowledgments, and return to the idle state upon completion of a communication transaction.

The modular implementation of the slave controller simplifies verification and improves maintainability by separating protocol functions into individual logical blocks. The detailed operation of each communication state, including address reception, acknowledgment generation, data reception, and data transmission, is governed by the slave finite state machine described in Section 3.6.

3.5 Master Finite State Machine

The operation of the master controller is governed by a finite state machine (FSM), which coordinates every stage of an I²C communication transaction. The FSM ensures that all protocol operations are executed in the correct sequence while maintaining compliance with the timing and control requirements of the I²C specification.

The master FSM is responsible for initiating communication, generating START and STOP conditions, transmitting the slave address and data bytes, receiving acknowledgments, handling read transactions, and resolving arbitration when multiple masters attempt to access the communication bus simultaneously. By dividing the communication process into a series of well-defined states, the controller achieves a modular and organized implementation that simplifies debugging and future modifications.

The master finite state machine consists of the following states:

3.5.1 IDLE State

The IDLE state represents the default operating condition of the master controller. In this state, both SDA and SCL remain released, allowing the pull-up resistors to maintain the bus at a logic high level. The controller continuously monitors the external ****start**** signal while waiting for a request to initiate communication. When a valid start request is received, the FSM transitions to the START state.

3.5.2 START State

In the START state, the master generates the I²C START condition by pulling the SDA line low while the SCL line remains high. This indicates the beginning of a new communication transaction and informs all slave devices to begin monitoring the incoming address frame. After successfully generating the START condition, the slave address together with the Read/Write bit is loaded into the transmission register, and the FSM proceeds to the SEND_ADDR state.

3.5.3 SEND_ADDR State

During the SEND_ADDR state, the master serially transmits the address frame onto the SDA line while simultaneously generating the clock pulses on the SCL line. One bit is transmitted during each clock cycle until the complete address frame has been sent. Throughout this process, the arbitration logic continuously compares the transmitted data with the actual state of the SDA line to detect any loss of bus ownership.

Once the complete address frame has been transmitted, the FSM transitions to the ADDR_ACK state.

3.5.4 ADDR_ACK State

After transmitting the address frame, the master releases the SDA line during the ninth clock pulse, allowing the addressed slave to generate an acknowledgment. The controller samples the SDA line to determine whether an ACK has been received.

If a valid acknowledgment is detected, the controller examines the Read/Write bit to determine the direction of communication. For a write request, the FSM transitions to the SEND_DATA state. For a read request, it transitions to the REC_DATA state. If no acknowledgment is received within the specified timeout period, the transaction is terminated by transitioning to the STOP state.

3.5.5 SEND_DATA State

During a write transaction, the SEND_DATA state serially transmits the contents of the data register onto the SDA line. Data bits are shifted out one bit at a time under the control of the generated SCL clock. Arbitration monitoring continues throughout the transmission to ensure that another master has not taken control of the communication bus.

After all data bits have been transmitted, the FSM proceeds to the DATA_ACK state.

3.5.6 DATA_ACK State

The DATA_ACK state allows the addressed slave to acknowledge successful reception of the transmitted data byte. The master releases the SDA line and samples the acknowledgment bit during the ninth clock pulse.

If an acknowledgment is received, the communication is considered successful and the FSM transitions to the STOP state. If no acknowledgment is detected, the transaction is also terminated by entering the STOP state.

3.5.7 REC_DATA State

The REC_DATA state is entered during a read transaction. In this state, the master releases the SDA line, allowing the addressed slave to transmit data onto the communication bus. The master samples the incoming data bits on the appropriate clock edge and reconstructs the received byte within an internal receive register.

Once all eight data bits have been received, the FSM transitions to the MASTER_ACK state.

3.5.8 MASTER_ACK State

After receiving a complete data byte, the master generates the appropriate response to the slave. If additional data bytes are required, the master transmits an ACK signal to request further transmission. If the received byte is the final byte, the master generates a NACK to indicate the end of the read operation before proceeding to the STOP state.

3.5.9 WAIT_BUS_FREE State

The WAIT_BUS_FREE state is entered whenever the master loses arbitration during address or data transmission. Upon detecting arbitration loss, the controller immediately releases both SDA and SCL, allowing the winning master to continue communication without interference.

The controller continuously monitors the communication bus while in this state. Once both SDA and SCL return to their idle high state, indicating that the bus has become free, the master may initiate a new communication transaction.

3.5.10 STOP State

The STOP state marks the completion of an I²C transaction. The master generates the STOP condition by allowing the SDA line to transition from logic low to logic high while the SCL line remains high. This releases the communication bus and returns it to the idle state, making it available for future transactions. After generating the STOP condition, the FSM returns to the IDLE state to await the next communication request.

3.6 Slave Finite State Machine

The slave controller is implemented using a finite state machine (FSM) that manages all operations performed by the slave during an I²C communication transaction. Unlike the master, the slave does not initiate communication or generate the clock signal. Instead, it continuously monitors the shared communication bus, waiting for a valid START condition generated by a master. Once communication begins, the FSM coordinates address reception, acknowledgment generation, data reception, data transmission, and termination of the transaction.

The slave finite state machine consists of the following states:

3.6.1 IDLE State

The IDLE state is the default operating condition of the slave controller. In this state, the controller continuously monitors the SDA and SCL lines for a valid START condition while keeping the SDA line in a high-impedance state. No communication is performed until a START condition is detected.

3.6.2 REC_ADDR State

After detecting a START condition, the FSM enters the REC_ADDR state. In this state, the slave receives the address frame transmitted by the master. The incoming serial bits are sampled on the rising edge of the SCL signal and shifted into an internal address register. Once the complete address frame has been received, the controller proceeds to verify whether the received address matches its configured slave address.

3.6.3 ADDR_ACK State

The ADDR_ACK state is responsible for acknowledging the received address. If the received address matches the configured slave address, the slave pulls the SDA line low during the ninth clock pulse to generate an ACK signal. At the same time, the controller examines the Read/Write (R/W) bit to determine the direction of communication.

If the R/W bit indicates a write operation, the FSM prepares to receive data. If the R/W bit indicates a read operation, the controller prepares to transmit data back to the master.

3.6.4 ACK_WAIT State

The ACK_WAIT state provides the necessary synchronization between the address acknowledgment phase and the beginning of data transfer. The controller waits for the completion of the acknowledgment clock cycle before transitioning

to either the data reception or data transmission state. This ensures that communication remains synchronized with the master's clock.

3.6.5 REC_DATA State

During a write transaction, the slave enters the REC_DATA state. In this state, incoming data bits are sampled on the rising edge of the SCL signal and shifted into the internal data register. After all eight bits have been received successfully, the FSM transitions to the DATA_ACK state.

3.6.6 SEND_DATA State

During a read transaction, the slave enters the SEND_DATA state. The data stored in the internal transmit register is shifted onto the SDA line one bit at a time while the master generates the clock pulses. The slave actively drives the SDA line only when transmitting a logic low and otherwise releases the line to preserve the open-drain characteristics of the I²C bus.

After transmitting the complete data byte, the FSM transitions to the WAIT_M_ACK state.

3.6.7 DATA_ACK State

The DATA_ACK state is entered after a complete data byte has been received during a write operation. The slave generates an acknowledgment by pulling the SDA line low during the ninth clock pulse, informing the master that the data has been successfully received and stored. After the acknowledgment has been transmitted, the controller returns to the IDLE state or waits for the next communication sequence depending on the transaction.

3.6.8 WAIT_M_ACK State

The WAIT_M_ACK state is used during read operations after the slave has transmitted a complete data byte. The slave releases the SDA line and waits for the master to transmit either an ACK or a NACK during the ninth clock pulse. An ACK indicates that the master wishes to continue receiving additional data, whereas a NACK indicates that the read operation has been completed. After receiving the master's response, the slave terminates the communication and returns to the IDLE state.

Throughout all communication states, the slave controller continuously monitors the bus for STOP conditions. Detection of a STOP condition immediately terminates the ongoing transaction and returns the FSM to the IDLE state, ensuring proper synchronization with future communication requests.

The slave finite state machine provides an organized framework for implementing the complete behavior of the I²C slave controller. By separating address reception, acknowledgment handling, data transfer, and transaction termination into dedicated states, the controller achieves reliable protocol-compliant communication while maintaining a modular and easily maintainable design.

3.7 Arbitration Logic

One of the most significant features of the implemented I²C controller is its support for multi-master communication through hardware arbitration. Since multiple master devices share the same communication bus, it is possible for two masters to initiate communication simultaneously. Without an arbitration mechanism, such situations would result in bus contention and data corruption. Therefore, the I²C protocol incorporates a non-destructive arbitration scheme that allows one master to continue transmission while the other automatically withdraws from the bus.

The arbitration mechanism relies on the open-drain architecture of the SDA line. Since no device actively drives the bus to a logic high level, any device may pull the line low without causing electrical conflicts. As a result, a logic '0' is considered dominant over a logic '1'. During every transmitted bit, each master continuously monitors the actual state of the SDA line while simultaneously transmitting its own data.

If a master transmits a logic '0' and observes a logic '0' on the bus, communication proceeds normally. Likewise, if a master transmits a logic '1' and observes a logic '1', no arbitration conflict exists. However, if a master attempts to transmit a logic '1' but detects a logic '0' on the SDA line, it concludes that another master is simultaneously transmitting a dominant bit. In this situation, the transmitting master immediately declares arbitration loss.

The implemented controller performs arbitration during both the address transmission phase and the data transmission phase. For every transmitted bit, the controller compares the intended transmit bit with the actual bus value. Whenever the controller detects that it has transmitted a recessive logic '1' while the communication bus is at a dominant logic '0', an internal arbitration loss signal is asserted.

The arbitration decision is implemented using the following logic:

```
if (tx_bit && !sda)
    arb_lost <= 1;
```

Here, tx_bit represents the value the master intends to transmit, while sda represents the actual logic level present on the shared communication bus. The arbitration condition is

therefore satisfied only when the transmitted value is logic '1' and the observed bus value is logic '0'.

Once arbitration loss is detected, the master immediately releases control of both the SDA and SCL lines by placing them in the high-impedance state. This prevents interference with the communication initiated by the winning master and preserves the integrity of the ongoing transaction. The losing master then enters the WAIT_BUS_FREE state, where it continuously monitors the communication bus until both SDA and SCL return to their idle high state.

After the current transaction has been completed and the bus becomes available, the losing master may attempt to initiate communication again. Since arbitration occurs on a bit-by-bit basis without interrupting the successful transmission, the I²C arbitration mechanism is referred to as non-destructive arbitration. The winning master continues communication without being aware that another master attempted to access the bus.

The implemented arbitration logic was verified by initiating simultaneous communication requests from both master controllers while transmitting different slave addresses and data values. Simulation results confirmed that the master transmitting the dominant bit retained ownership of the communication bus, whereas the competing master correctly detected arbitration loss, released the bus, and waited until the transaction had completed before attempting retransmission.

The successful implementation of this arbitration mechanism demonstrates compliance with one of the most important features of the I²C protocol and enables reliable communication in a multi-master environment without the need for external bus management circuitry.

3.8 Read and Write Operations

The implemented I²C controller supports both write and read transactions, enabling bidirectional communication between the master and slave devices. The direction of communication is determined by the Read/Write (R/W) bit transmitted as the least significant bit of the address frame. Depending on the value of this bit, both the master and slave controllers transition into different operating states to either transmit or receive data.

3.8.1 Write Operation

During a write operation, the master initiates communication by generating a START condition and transmitting the slave address with the R/W bit set to logic '0'. After receiving the complete address frame, the addressed slave compares the

received address with its configured address. If a match is detected, the slave generates an acknowledgment during the ninth clock pulse.

Following successful address acknowledgment, the master serially transmits the data byte onto the SDA line while simultaneously generating the required clock pulses on the SCL line. The slave samples each incoming bit on the rising edge of the clock and reconstructs the complete data byte within its internal data register. Once all eight bits have been received successfully, the slave generates another acknowledgment to indicate successful reception of the transmitted data.

After receiving the acknowledgment, the master terminates the transaction by generating a STOP condition, thereby releasing the communication bus for future transactions.

The sequence of operations during a write transaction is summarized below:

START → Address + Write (R/W = 0) → ACK → Data Byte → ACK → STOP

3.8.2 Read Operation

During a read operation, the master again initiates communication by generating a START condition. However, in this case the address frame is transmitted with the R/W bit set to logic '1', indicating that the slave is requested to transmit data.

After successfully acknowledging the address, the slave loads the previously stored data into its transmit register and assumes control of the SDA line. The master releases the SDA line and continues generating the clock signal while sampling each transmitted bit. As the clock pulses are generated, the slave serially shifts the data onto the communication bus until the complete byte has been transmitted.

Upon receiving the complete data byte, the master reconstructs the received information within its internal receive register. Depending on the communication requirements, the master may generate an ACK to request additional data or a NACK to indicate that no further bytes are required. In the present implementation, a single-byte read transaction is performed, after which the master terminates communication by generating a STOP condition.

The sequence of operations during a read transaction is summarized below:

START → Address + Read (R/W = 1) → ACK → Data Byte ← Slave → Master
ACK/NACK → STOP

3.8.3 Bidirectional Data Transfer

Support for both read and write transactions significantly enhances the functionality of the implemented I²C controller. Unlike a write-only implementation, the developed controller allows data to flow in either direction depending on the value of the R/W bit. During write operations, the master functions as the transmitter while the slave acts as the receiver. During read operations, these roles are reversed, with the slave transmitting data and the master receiving it.

To support this behavior, additional finite state machine states were incorporated into both controllers. The master controller includes dedicated receive and acknowledgment states for collecting data transmitted by the slave, while the slave controller includes dedicated transmit states that enable it to drive the SDA line during read transactions. This coordinated operation allows seamless switching of bus ownership while maintaining compliance with the I²C protocol.

Simulation results confirmed the correct operation of both communication modes. Write transactions successfully stored data within the addressed slave, while subsequent read transactions returned the stored data to the requesting master without communication errors. These results demonstrate the successful implementation of complete bidirectional communication within the proposed multi-master, multi-slave I²C system.

3.9 Simulation Environment

To verify the functionality of the proposed I²C controller, the complete design was simulated using the Xilinx Vivado Design Suite. Simulation was performed prior to hardware implementation to validate the correctness of the protocol, verify communication between the master and slave controllers, and identify any timing or functional errors in the design.

A dedicated Verilog testbench was developed to generate the system clock, reset signal, transaction requests, target slave addresses, and data values required to evaluate the operation of the controller. The testbench provided controlled stimulus to both master controllers, enabling verification of individual transactions as well as simultaneous communication attempts in a multi-master environment.

The simulation environment consisted of the following functional modules:

- Top-level I²C module
- Two master controllers
- Two slave controllers
- Shared SDA and SCL communication bus

- Verilog testbench
- Vivado simulation waveform viewer

Multiple simulation scenarios were executed to evaluate different aspects of the implemented design. Initially, individual write transactions were performed to verify address transmission, acknowledgment generation, and successful data storage within the addressed slave. Read transactions were then carried out to confirm correct implementation of bidirectional communication, ensuring that previously stored data could be transmitted back to the requesting master.

The arbitration mechanism was verified by initiating simultaneous transmission requests from both master controllers. The simulation confirmed that the controller correctly detected arbitration loss, released the communication bus, and allowed the winning master to complete the transaction without interruption. Additional simulations were performed to verify START and STOP condition detection, address matching, ACK/NACK handling, and correct operation of both slave devices.

Waveforms generated by the Vivado simulator were used to observe the behavior of internal signals, including finite state machine transitions, SDA and SCL activity, address transmission, data transfer, acknowledgment signals, arbitration status, and read/write operations. These waveforms provided detailed insight into the operation of the controller and were used to confirm compliance with the I²C protocol.

The simulation results presented in the following chapter demonstrate that the implemented design successfully performs protocol-compliant communication, supports bidirectional data transfer, and correctly handles multi-master arbitration under various operating conditions.

Chapter 4: Simulation Results and Verification

4.1 Introduction

The functionality of the proposed I²C communication system was verified through behavioral simulation using the Xilinx Vivado Design Suite. The objective of the verification process was to ensure that every functional block operated according to the I²C protocol specification and that reliable communication was achieved between the master and slave devices under various operating conditions.

Simulation was carried out using a dedicated Verilog testbench that generated the required clock, reset, transaction requests, slave addresses, and data values. A series of test cases was developed to evaluate different aspects of the implemented design, including write transactions, read transactions, address matching, acknowledgment generation, START and STOP condition detection, multi-master arbitration, and communication with multiple slave devices.

For each test case, the internal controller states, SDA and SCL bus activity, transmitted addresses, data bytes, acknowledgment signals, and arbitration status were observed using the Vivado waveform viewer. The obtained simulation results were compared with the expected behavior defined by the I²C protocol to verify the correctness of the implementation.

The following sections present the simulation results for each test case along with an analysis of the observed waveforms.

4.2 Write Transaction Verification

The first verification test was performed to validate the write operation of the implemented I²C controller. The objective of this test was to confirm that the master successfully transmitted the slave address and data byte, that the addressed slave correctly recognized its address, generated the required acknowledgment signals, and stored the transmitted data.

During the simulation, the master controller initiated communication by generating a START condition on the shared communication bus. Following the START condition, the master transmitted the 7-bit slave address together with the Read/Write (R/W) bit set to logic '0', indicating a write operation. The addressed slave continuously monitored the SDA and SCL lines, compared the received address with its configured address, and generated an acknowledgment after a successful address match.

After the address acknowledgment, the master transmitted an 8-bit data byte serially over the SDA line while simultaneously generating the clock pulses on the SCL line. The slave sampled each incoming bit on the rising edge of the clock signal and reconstructed the transmitted byte within its internal data register. Upon successful reception of the complete data byte, the slave generated a second acknowledgment, confirming that the data had been received correctly.

The waveform shown in Figure 4.1 demonstrates the successful execution of the complete write transaction. The waveform clearly shows the generation of the START condition, address transmission, acknowledgment from the slave, data transmission, data acknowledgment, and the STOP condition marking the end of the communication session.

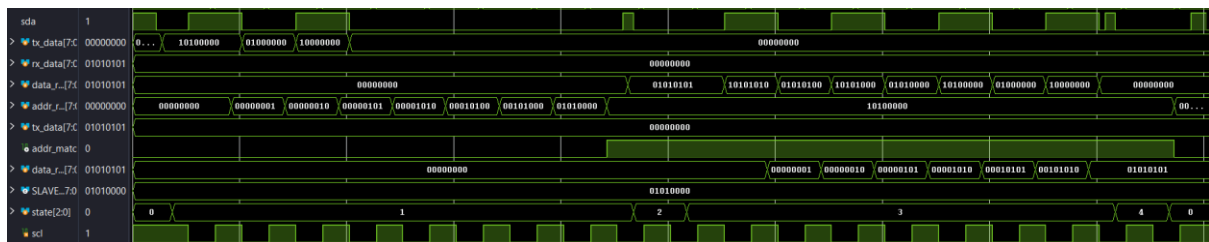


Figure 4.1

The internal state transitions of both the master and slave controllers were also verified during the simulation. The master successfully progressed through the START, SEND_ADDR, ADDR_ACK, SEND_DATA, DATA_ACK, and STOP states, while the slave transitioned through the IDLE, REC_ADDR, ADDR_ACK, REC_DATA, and DATA_ACK states. These state transitions confirm that both controllers operated according to the implemented finite state machines.

Furthermore, the transmitted data was successfully stored in the slave's internal data register without any communication errors. The SDA and SCL signals followed the expected timing relationships defined by the I²C protocol, and all acknowledgment signals were generated during the correct clock cycles.

The simulation results verify that the implemented write operation is fully functional and conforms to the standard I²C communication protocol.

Result: The write transaction was successfully completed, with correct address recognition, acknowledgment generation, data transfer, and data storage within the addressed slave.

4.3 Read Transaction Verification

The read transaction was verified to ensure that the implemented I²C controller correctly supports bidirectional communication. The objective of this test was to confirm that the

addressed slave successfully transmitted previously stored data to the requesting master while complying with the I²C protocol.

The simulation began with the master initiating communication by generating a START condition, followed by transmission of the slave address with the Read/Write (R/W) bit set to logic '1', indicating a read request. After receiving the address frame, the addressed slave compared the received address with its configured slave address and generated an acknowledgment upon a successful address match.

Following the acknowledgment phase, the slave assumed control of the SDA line and began transmitting the contents of its internal data register to the master. During this period, the master released the SDA line and continued generating the clock signal on the SCL line while sampling the incoming data bits on the rising edge of the clock. After receiving the complete data byte, the master reconstructed the transmitted value within its receive register and generated the appropriate acknowledgment before terminating the communication with a STOP condition.

The waveform shown in Figure 4.2 illustrates the successful execution of the read transaction. It can be observed that bus ownership of the SDA line is correctly transferred from the master to the addressed slave after the address acknowledgment. The slave serially transmits the stored data, while the master receives each bit without communication errors.



Figure 4.2

The finite state machines of both controllers also exhibited the expected behavior during the simulation. The master progressed through the START, SEND_ADDR, ADDR_ACK, REC_DATA, MASTER_ACK, and STOP states, while the slave transitioned through the IDLE, REC_ADDR, ADDR_ACK, SEND_DATA, and WAIT_M_ACK states. These state transitions confirm the correct implementation of bidirectional communication within the controller.

The received data stored in the master's receive register was verified against the data previously written to the slave. The values matched exactly, confirming that the slave successfully transmitted the stored data and that the master correctly reconstructed the received byte.

The simulation therefore verifies that the implemented read operation functions correctly and enables reliable bidirectional communication between the master and slave devices.

Result: The read transaction was successfully completed. The slave correctly transmitted the stored data, the master accurately received the data, and the communication complied with the timing and operational requirements of the I²C protocol.

4.4 Multi-Master Arbitration Verification

The multi-master arbitration test was conducted to verify the controller's ability to correctly resolve simultaneous communication requests from multiple master devices. The objective of this test was to ensure that bus contention was handled according to the I²C protocol without causing data corruption or communication failure.

To perform this verification, both master controllers were instructed to initiate communication at approximately the same time while attempting to access the shared SDA and SCL bus. Since both masters transmitted different address or data bits during the transaction, an arbitration condition was created. Throughout the transmission, each master continuously monitored the actual state of the SDA line while comparing it with the value it intended to transmit.

Whenever a master attempted to transmit a logic '1' but detected a logic '0' on the SDA line, the arbitration logic identified that another master was transmitting a dominant bit. The controller immediately asserted the arbitration loss signal (`arb_lost`), released control of both SDA and SCL by entering the high-impedance state, and transitioned to the `WAIT_BUS_FREE` state. This allowed the winning master to continue communication without interruption, preserving the integrity of the transaction.

The waveform shown in Figure 4.3 demonstrates the arbitration process. It can be observed that one master continues transmitting while the competing master detects arbitration loss and immediately stops driving the communication bus. The losing master remains inactive until the bus returns to the idle state, after which it becomes eligible to initiate another transaction.

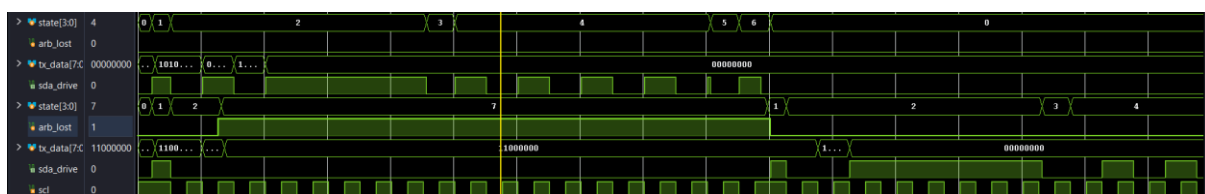


Figure 4.3

The simulation also confirmed that arbitration was performed without corrupting the transmitted address or data. The winning master completed the communication successfully, while the losing master correctly relinquished control of the bus. This behavior demonstrates the non-destructive arbitration mechanism defined by the I²C

specification, where communication continues seamlessly despite simultaneous transmission attempts.

Internal control signals, including the arbitration loss flag and finite state machine transitions, were monitored throughout the simulation. The observed behavior matched the expected operation of the implemented arbitration logic, confirming that bus ownership was correctly determined on a bit-by-bit basis.

The successful completion of this test verifies that the implemented controller supports reliable multi-master communication and correctly resolves simultaneous bus access requests without requiring any external arbitration circuitry.

Result: The arbitration mechanism successfully identified bus contention, allowed the winning master to complete the transaction, forced the losing master to release the communication bus, and maintained error-free communication throughout the transaction.

4.5 Multi-Slave Communication Verification

The multi-slave communication test was conducted to verify that the implemented I²C controller correctly communicates with multiple slave devices connected to the same communication bus. The objective of this test was to ensure that only the slave whose address matches the transmitted address responds to the master's request, while all other slave devices remain inactive throughout the transaction.

During the simulation, two slave controllers were instantiated with different 7-bit addresses. Separate communication transactions were then initiated by the master using the address of each slave. For every transaction, both slave controllers continuously monitored the SDA and SCL lines and independently compared the received address frame with their configured addresses.

When the transmitted address matched the address of one of the slave controllers, only that slave generated an acknowledgment and participated in the communication. The second slave remained in the idle state, neither driving the SDA line nor responding to the transaction. This behavior prevented bus contention and ensured that communication occurred exclusively between the master and the addressed slave.

The simulation also verified that data transmitted during write operations was stored within the correct slave device and that subsequent read operations returned the corresponding stored data. Throughout the verification process, the non-addressed slave remained inactive, confirming that address matching logic functioned correctly and prevented unintended communication.

The successful execution of these tests demonstrates that the implemented controller supports reliable communication with multiple slave devices sharing the same I²C bus while ensuring that only the addressed slave participates in each transaction.

Result: Communication with multiple slave devices was successfully verified. Each slave responded only to its assigned address, all acknowledgments were generated correctly, and data was transferred exclusively between the master and the addressed slave.

4.6 START/STOP Detection and ACK/NACK Verification

The final verification test was conducted to ensure the correct implementation of the START and STOP conditions together with the acknowledgment (ACK) and Not Acknowledgment (NACK) mechanisms defined by the I²C protocol. These protocol elements are essential for establishing, maintaining, and terminating reliable communication between the master and slave devices.

At the beginning of every transaction, the master successfully generated a START condition by driving the SDA line from a logic high state to a logic low state while the SCL line remained high. Both slave controllers continuously monitored the communication bus and correctly detected the START condition, enabling them to begin receiving the incoming address frame.

After transmission of the slave address, the addressed slave compared the received address with its configured address. Upon detecting a valid address match, the slave generated an acknowledgment by pulling the SDA line low during the ninth clock pulse. The master successfully sampled this acknowledgment and continued with the communication sequence. Following data transmission, the slave generated a second acknowledgment to confirm successful reception of the data byte. These acknowledgments ensured reliable communication between the master and the addressed slave.

For read transactions, the master correctly acknowledged the received data after the slave completed data transmission. When the communication was complete, the master terminated the transaction by generating a STOP condition. This was achieved by allowing the SDA line to transition from logic low to logic high while the SCL line remained high. Both slave controllers correctly detected the STOP condition and returned to the IDLE state, releasing the communication bus for subsequent transactions.

Successful verification of these protocol operations confirms that the controller can reliably establish communication, exchange acknowledgment signals, and terminate transactions without protocol violations.

Result: START and STOP conditions were generated and detected correctly, ACK signals were exchanged successfully after address and data transmission, and all communication sessions were terminated in accordance with the I²C protocol.

4.7 Functional Summary

A comprehensive set of simulations was performed to evaluate the functionality of the proposed I²C controller under various operating conditions. The verification process confirmed that all major features of the design operate correctly and comply with the expected behavior of the I²C communication protocol.

The implemented controller successfully established communication between the master and slave devices through the correct generation and detection of START and STOP conditions. Address transmission and recognition were verified, ensuring that only the intended slave responded to a given transaction. The acknowledgment mechanism also functioned correctly, with ACK signals being generated and detected at the appropriate stages of communication.

Both write and read transactions were successfully executed. During write operations, the master transmitted data that was accurately received and stored by the addressed slave. During read operations, the slave correctly transmitted the stored data back to the master, demonstrating successful bidirectional communication and proper implementation of the Read/Write (R/W) bit functionality.

The multi-master arbitration mechanism was successfully validated by initiating simultaneous communication requests from two master controllers. The arbitration logic correctly identified the winning and losing masters, allowing uninterrupted communication while preventing bus contention and data corruption. Furthermore, communication with multiple slave devices was verified, confirming that each slave responded only to its assigned address while the remaining slave remained inactive.

Overall, the simulation results demonstrate that the proposed design achieves reliable protocol-compliant communication over a shared I²C bus. The modular implementation of the master controller, slave controller, arbitration logic, and open-drain bus architecture provides a robust foundation for FPGA-based I²C communication systems.

Table 4.6 summarizes the outcome of the functional verification performed during this project.

Chapter 5: Conclusion and Future Work

5.1 Conclusion

This project successfully achieved the design, implementation, and functional verification of a multi-master, multi-slave I²C communication system using Verilog HDL. The developed controller adheres to the fundamental principles of the I²C protocol and demonstrates reliable communication over a shared two-wire bus.

The implemented design supports two independent master controllers and two slave controllers connected through common SDA and SCL communication lines. Core protocol features, including START and STOP condition generation, 7-bit addressing, acknowledgment (ACK/NACK) handling, address matching, open-drain bus operation, and bidirectional read and write communication, were successfully implemented and verified. In addition, a non-destructive arbitration mechanism was incorporated to resolve simultaneous bus access requests from multiple masters, ensuring reliable communication without data corruption.

Behavioral simulation using Xilinx Vivado confirmed the correct operation of all functional modules. The verification results demonstrated successful write transactions, read transactions, communication with multiple slave devices, proper acknowledgment generation, accurate START and STOP detection, and correct arbitration during simultaneous master transmissions. The observed simulation waveforms closely matched the expected behavior defined by the I²C protocol specification, confirming the correctness of the implemented design.

Throughout the development process, valuable practical experience was gained in digital system design, finite state machine development, shared-bus communication, protocol implementation, and simulation-based verification. The project also provided insight into debugging complex hardware designs, particularly in areas such as arbitration timing, acknowledgment handling, and bidirectional data transfer.

Overall, the proposed I²C controller provides a modular, reliable, and protocol-compliant communication system suitable for FPGA-based embedded applications. The modular architecture of the design also allows additional features to be incorporated with minimal modifications, making it a strong foundation for future enhancements and more advanced communication systems.

5.2 Future Work

Although the implemented I²C controller successfully supports multi-master, multi-slave communication with bidirectional data transfer and arbitration, several enhancements can be incorporated to further improve its functionality and increase its applicability in practical embedded systems.

One possible enhancement is the implementation of repeated START conditions, allowing multiple transactions to occur without releasing the communication bus. This feature is commonly used when switching between write and read operations during communication with memory devices and sensors.

Another improvement would be the addition of clock stretching support. In the standard I²C protocol, slave devices may temporarily hold the SCL line low to delay communication until they are ready to continue. Incorporating this feature would improve compatibility with a wider range of commercial I²C peripherals.

The current implementation supports the transfer of a single data byte per transaction. Future versions of the controller can be extended to support multi-byte data transfers by incorporating byte counters and automatic sequential transmission. This enhancement would significantly improve communication efficiency for applications involving larger amounts of data.

Support for 10-bit slave addressing can also be added to increase the number of addressable devices on the communication bus. This would allow the controller to communicate with peripherals that require extended addressing beyond the standard 7-bit format.

The project was verified through behavioral simulation using Xilinx Vivado. As a future enhancement, the design can be synthesized and deployed on an FPGA development board to validate its performance under real hardware conditions. Hardware implementation would allow evaluation of signal integrity, timing behavior, and communication with external I²C devices such as EEPROMs, temperature sensors, real-time clocks, and display modules.

Additional improvements may include integrating the I²C controller with standard on-chip bus interfaces such as AXI-Lite or Wishbone, enabling seamless communication with embedded processors in System-on-Chip (SoC) designs. Features such as FIFO-based buffering, interrupt generation, configurable clock frequencies, programmable slave addresses, and error detection mechanisms can also be incorporated to improve system performance and flexibility.

These enhancements would transform the current implementation from a functionally verified educational design into a more feature-complete and industry-oriented I²C controller suitable for integration into complex FPGA and embedded system applications.